Writing in the Air: Measuring and Mitigating Domain Shift in Handwritten Digit Recognition

Programming with Python, Fall 2026 — Course Project Proposal
Group members: Name 1, Name 2, Name 3, Name 4

1. Introduction and Problem Statement

Convolutional neural networks reach above 99% accuracy on the MNIST handwritten digit benchmark, which is often taken to mean the task is solved. In practice, a model deployed in the real world rarely sees the same kind of input it was trained on. This gap between training and deployment distributions is known as *domain shift*, and it is one of the most common reasons why machine learning systems that look excellent on paper fail once they are used.

This project studies domain shift on a concrete, self-collected example. We build a gesture-controlled virtual whiteboard: a webcam tracks the user's hand, an extended index finger draws, and a closed fist stops drawing. Digits written in mid-air differ systematically from MNIST digits — the stroke is thicker and of uneven width, the hand trembles, proportions are distorted, and the user cannot lift the pen, so unwanted connecting strokes appear. We quantify how much accuracy is lost on such data and determine which preprocessing and training techniques recover it.

Research question. How large is the accuracy drop when an MNIST-trained classifier is applied to digits written in the air, and which interventions — stroke normalisation, centre-of-mass alignment, training-time augmentation, or fine-tuning — close that gap most effectively?

2. Data

Source domain. MNIST (LeCun et al., 1998), 70,000 grayscale 28×28 images, accessed through `torchvision`.

Target domain (collected by us). Each of the four group members writes 75–100 digits in the air using our whiteboard application, giving roughly 300–400 labelled images in total, stored together with the identity of the writer. Images are saved in raw form, without resizing, so that every preprocessing variant can be evaluated on identical source data. Members are instructed to write naturally rather than to imitate the MNIST style, since the point of the study is to capture the real difference between the two domains. This dataset is an original contribution and does not exist anywhere else.

3. Method

Acquisition. MediaPipe Hands returns 21 hand landmarks per frame. A finger counts as extended when its tip lies above its PIP joint; the resulting five-element state vector selects the drawing mode (index only → draw, fist → pause, open palm → clear). Raw fingertip positions are jittery, so the drawing point is smoothed over a short sliding window, and the camera image is mirrored horizontally to make the interaction intuitive.

Classifier. A compact CNN (three Conv–BatchNorm–ReLU–MaxPool blocks, dropout, one dense layer) trained on MNIST. A multinomial logistic regression on raw pixels serves as a baseline, establishing how much of the task is attributable to the convolutional architecture itself rather than to the data.

Interventions. Each is implemented as an independent, switchable pipeline step so that any combination can be evaluated: stroke-thickness normalisation by morphological operations; centre-of-mass alignment and aspect-preserving rescaling that reproduces the original MNIST normalisation procedure; training-time augmentation with random dilation, rotation, translation and elastic deformation; and fine-tuning on a held-out portion of our own data.

4. Experiments and Evaluation

The evaluation metrics are accuracy, macro-F1 and per-class F1, together with confusion matrices. Every configuration is run with three random seeds and reported as mean ± standard deviation, so that differences between variants can be distinguished from run-to-run noise.

Experiment	Purpose
Baseline vs. CNN on MNIST	Establish source-domain performance and verify the training setup.
MNIST model on our data, minimal preprocessing	Measure the raw size of the domain gap — the central result.
Ablation over preprocessing steps	Isolate the contribution of stroke normalisation and of centring.
Augmented training	Test whether the gap can be closed without ever seeing target data.
Fine-tuning on 70% of our data	Upper bound: how much a small amount of in-domain data buys.
Leave-one-writer-out	Does the model generalise to a writing style it has not seen?

Visualisations. Learning curves; confusion matrices for both domains; a bar chart of accuracy per ablation variant with error bars; side-by-side samples of the same digit from MNIST and from each writer; distributions of stroke thickness and image fill ratio across the two domains; and galleries of misclassified examples.

Reproducibility. All random seeds are fixed, library versions are pinned in `requirements.txt`, and every number quoted in the report is written to disk by the same script that produces the figures. No result is reported that the code does not generate.

5. Expected Outcome and Contribution

We expect a substantial accuracy drop on in-air digits relative to MNIST, with confusions concentrated in pairs whose distinction depends on fine stroke detail. We expect geometric normalisation to recover a noticeable part of the gap at zero training cost, augmentation to help further without requiring any target data, and fine-tuning on a small sample to perform best. The leave-one-writer-out experiment should indicate how much of the remaining error is writer-specific rather than domain-wide.

The contribution is not a new architecture but an empirical study on an original dataset: a quantified measurement of a practically relevant failure mode, and a ranking of mitigations by the accuracy each one actually recovers.

6. Tools, Division of Work and Schedule

Libraries. PyTorch and torchvision (model, data), OpenCV and MediaPipe (acquisition, image processing), NumPy, scikit-learn (baseline, metrics), Matplotlib (figures).

Member	Responsibility	Files
A	Acquisition: hand tracking, gesture logic, smoothing, data-collection pipeline; live demo	whiteboard.py, collect_data.py
B	Modelling: CNN architecture, training loop, baseline, fine-tuning	model.py, train.py, config.py
C	Image processing and evaluation: normalisation, augmentation, metrics, ablation runs	preprocessing.py, evaluate.py, ablation.py
D	Analysis, figures, cross-writer study, report and coordination	analysis.py, report

Data collection is performed jointly by all four members, which is what makes the cross-writer analysis possible. Interfaces between the modules are agreed before implementation begins, so the four parts can be developed in parallel and integrated at the end.

Schedule. Acquisition and MNIST training proceed in parallel on day one, followed by joint data collection; experiments and ablation on day two; report and presentation rehearsal on day three. Submission is due 13 November 2026.

7. References

[1] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
[2] F. Zhang et al. MediaPipe Hands: On-device Real-time Hand Tracking. arXiv:2006.10214, 2020.
[3] A. Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS*, 2019.
[4] G. Bradski. The OpenCV Library. *Dr. Dobb's Journal of Software Tools*, 2000.
[5] P. Simard, D. Steinkraus, J. Platt. Best Practices for Convolutional Neural Networks Applied to Visual Document Analysis. *ICDAR*, 2003.